Port Daddy Coordination Papers — Chapter II of VII

The Single-Writer Kernel:

A Local Transactional Reference Monitor for Agent Swarms

Erich Owens

Curiositech LLC

engineering@portdaddy.dev

August 2026
Version 1.1 (revised pre-print)

Abstract

A swarm of autonomous coding agents sharing one machine collides over the same scarce things: network ports, files on disk, mutual-exclusion locks, and the record of who did what. The instinct, trained on a decade of distributed-systems literature, is to reach for consensus — replicate the state, run an agreement protocol. We make the opposite, and we argue correct, move: collapse the entire problem onto a **single writer over a single local SQLite database in write-ahead-log (WAL) mode**, and let the operating system's file lock serialize every mutation. There is one decider, so there is no agreement to reach.

This paper presents that kernel — the **substrate layer** of a four-layer coordination stack — together with the implemented half of the coordination protocol that rides directly on it (a durable message bus, decaying stigmergic markers, violable commitments, a cryptographic delegation chain, and a policy monitor) as a **single-writer transactional reference monitor** in the sense of Anderson and Lampson: a small, tamper-evident mediator of every security-relevant operation, realized locally rather than as an abstract security kernel. We state the kernel's invariants as theorems — mutual exclusion, idempotence, the consistency model (serializable transactions, a linearizable claim history), oracle-bound obligation closure — and we are deliberate about where the promises stop. In particular we *split* durability by fault class: a write that returns success survives a *process* crash but is *not guaranteed* against power loss under the default WAL configuration; and we correct a widespread over-claim, showing that the policy monitor is a post-commit *detector*, not a pre-commit regimenter, with the only genuine pre-commit regimentation credited to a uniqueness constraint and a boot-time admission gate. The kernel is solid where it is local and provisional exactly where a cross-machine economy of agents would need it to become cryptographic and continuous — and we say so in the same words throughout. Companion chapters in the same series (*The Legible Swarm*, *From Spawn to Person*, *The Harbor Economy*) build the operator interface, the identity-and-reputation bridge, and the cross-machine market on the floor this paper specifies.

Keywords: reference monitor, single-writer transactions, SQLite write-ahead logging, durability

by fault class, linearizability, deontic logic, commitments, stigmergy, local-first coordination, multi-agent systems

Reading time: approximately 45 minutes end-to-end; the Reader's Map (§2) routes you to a 15-minute path and to single-section paths. This is Chapter II of a seven-chapter collection. The Legible Swarm *is the operator-facing entry point and a natural place to start;* From Spawn to Person *builds an identity-and-reputation bridge on top of this kernel;* The Harbor Economy *carries the cross-machine market, including cross-machine capability transfer — which belongs to the cross-machine layer and lives there, not here. Any chapter can be read first, but every later layer rests on this one.*

Contents

1 Where this paper sits, and what it promises

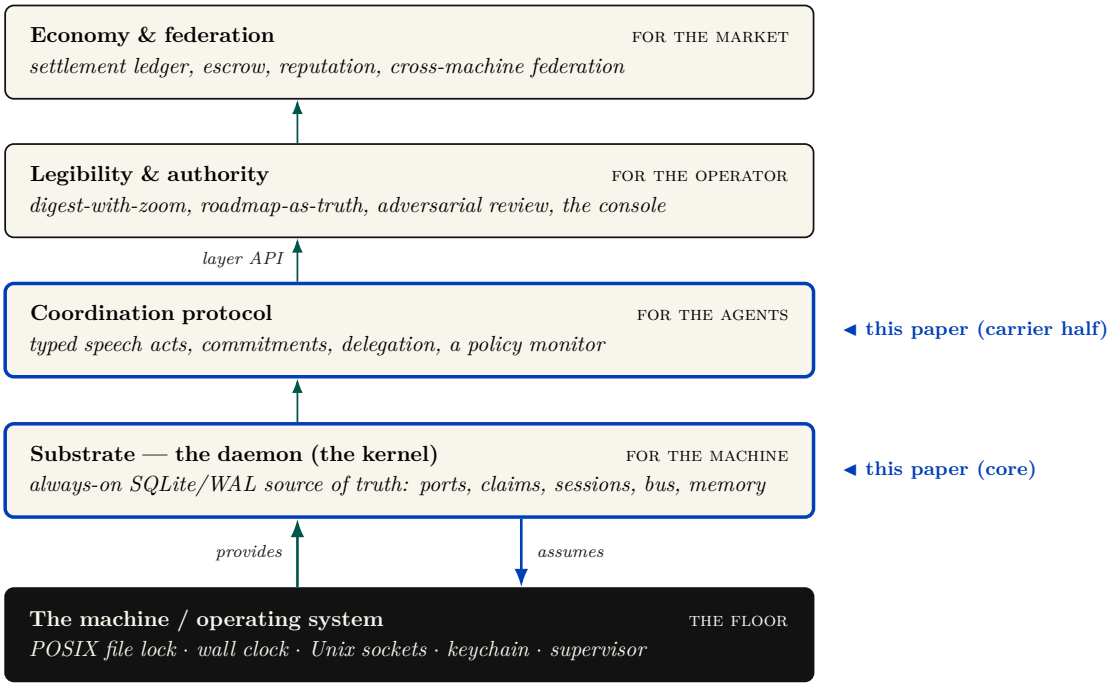

Figure 1: The four-layer coordination stack. Each layer serves a different *whom* (machine, agents, operator, market). **Chapter II owns the two lowest layers**: all of the substrate (the kernel) and the implemented carrier half of coordination (the message bus, stigmergic markers, commitments, the cryptographic authorization chain, the policy monitor); the coordination *semantics* (protocol state machines, the agent sandbox) and the whole legibility layer belong to Chapters I and IV (Chapter I: *The Legible Swarm*; Chapter IV: *The Harbor Economy* owns the market rung). From the machine the substrate *assumes* only an advisory file lock, one clock, and a supervisor; upward it *provides* atomic TTL'd claims, a durable bus, the deontic split, and an audit log. Later chapters build on or cross-cut this floor; chapter numbers are not layer numbers.

We model an agent-coordination system as a four-layer stack (Figure 1), each layer serving a different *whom*.

Substrate (the machine).
A single always-on daemon over one local database: durable storage and serialized mutation. *This paper's core.*

Coordination (the agents).
The protocol agents speak over the substrate: ownership claims, a message bus, stigmergic markers, obligations, and policy enforcement. This paper specifies the implemented half of it.

Legibility (the operator).
The human-facing view that renders the swarm intelligible. Treated in a accompanying chapter (*The Legible Swarm*).

Economy (the market between operators).
The cross-machine market in which operators trade work and reputation. Treated in a accompanying chapter (*The Harbor Economy*).

You climb the stack in order, and you never buy a layer before you need it. This paper is the load-bearing floor. The legibility layer reads the kernel's claims, bus, and policy monitor to

render the swarm; the personhood argument leans the entire notion of a *durable agent identity* on the kernel's continuity machinery; the economy's settlement ledger is, mechanically, another set of tables under the same single writer, and its cross-machine capability transfer rides the kernel's cryptographic delegation chain. If this floor is unsound, everything above it inherits the fault. So the discipline of this paper is to state each promise precisely and, where the promise has an edge, to draw the edge in ink.

1.1 The one-sentence thesis

> *The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database — the one process that mediates contended resources, and the one place where "true" is decided rather than asserted.*

That is the whole claim, and it has two halves, which are the kernel's entire job; everything else is convenience.

1. **Durability (with an edge):** a write that returned success is durable across the death of any agent — and, we will be careful, across the death of the *daemon process*, though *not* unconditionally across power loss.
2. **Mediation (with a correction):** a state the kernel *regiments* cannot come to exist; a state the kernel merely *enforces* is detected and compensated within a bounded window. These are different guarantees and we will never conflate them.

1.2 Why "reference monitor," and why *not* consensus

The right mental model is the **reference monitor** of Anderson and Lampson [1, 2]: a mediator that is *always invoked* (complete mediation), *tamper-evident*, and *small enough to verify*. The kernel instantiates this not as an abstract security kernel but as a concrete local daemon over one SQLite file. Complete mediation holds *by construction over the kernel's own state*: because every mutation of *coordination state* routes through one writer holding one file lock, the operating system's serialization *is* the mediation primitive — Saltzer and Schroeder's "economy of mechanism" [3] done with one moving part instead of N hand-rolled critical sections. We are deliberate about the scope of the word *complete*: it is complete over writes that *enter* the daemon, not over what a same-user agent can do *beside* it. A classical reference monitor earns "always invoked" from the hardware/operating-system boundary that forces every access through it; this kernel has no such boundary at the substrate layer, so a same-user process can mutate shared state the daemon never sees — edit a file under an advisory claim, run `git` directly, or write the SQLite file itself — without routing through the writer. Mediation is therefore complete over the daemon's *application-program interface* and advisory everywhere else; making it complete over the agent's host capabilities is an out-of-band enforcement problem we scope to the threat model (§13) and credit, when it lands, to a separate-user kernel boundary that does not yet exist (§13.2).

> **Key idea.** The swarm *looks* like a distributed-systems problem — many agents, contention, failover — so the trained reflex is Raft, Paxos, CRDTs. The kernel's defining move is to refuse that problem. One writer, one machine, one file: there is nothing to *agree* on, so the consensus impossibility of Fischer, Lynch, and Paterson [11] does not bite. It is sidestepped, not solved. Distribution is a cross-machine concern that belongs to the economy layer; pushing a consensus protocol down into the local substrate would be the wrong layer.

This is the single most important architectural decision a local-first coordination layer can make, and the field is littered with systems that got it wrong by manufacturing a consensus problem they did not have [9]. We will state the formal payoff of the choice as a conditional theorem in §11: when every read and write is routed through the sole daemon connection and responses follow commit, transactions are *serializable* and claim/lock operations are *linearizable* — the property that lets the coordination layer treat "who holds this" as ground truth without any agreement protocol at all.

1.3 A research-maturity scale, and why the grades are load-bearing

A systems paper that mixes "we proved this" with "we hope to build this" is unfalsifiable. So every load-bearing claim in this paper carries an explicit *maturity grade* on a four-point scale, plus two refinements introduced here because a single grade cannot formally carry a claim that spans fault classes or interception points. The grade is a property of the *claim*, not a marketing adjective: it says how far the claim is from a verified, running artifact. The concrete artifacts behind each grade are catalogued in Appendix A; the body argues at the level of mechanisms.

Table 1: The research-maturity scale used throughout. The last two rows are refinements this paper introduces (see §5 and §8).

Grade	Meaning
implemented	Realized, exercised, and true under the production runtime.
PARTIAL	Realized but partial: holds under a narrower condition than the prose might invite, or detects rather than prevents.
SPECIFIED	A concrete design exists, but no running artifact realizes it.
proposed	Named as a direction; no design yet.
I1a / I1b	Durability *split by fault class*: I1a (process-crash) is **implemented**; I1b (power-loss) is *not guaranteed* under the default WAL configuration. A claim that conflates fault classes cannot carry one grade.
regimented / enforced	Prohibition split by interception point: *regimented* = rejected pre-commit, truly unreachable (a uniqueness constraint / boot-admission gate); *enforced* = detected post-commit, halted or compensated within a bounded window (the policy monitor). "Physically unreachable" is reserved for the regimented set.

> **Pitfall.** The grades are not decoration. The two most consequential corrections in this paper — the durability split and the regimentation/detection split — are exactly the places where a single optimistic grade would let the kernel's foundational promise be read as stronger than the artifact delivers. When a system claims "the database survives every agent's death," a reader hears "power-loss durable." It is not, by default. Saying so is the difference between a systems paper and a brochure.

2 Reader's Map

Table 2: Reader's Map. Find your row; read those sections first.

If you are...	**... read first**	**... load-bearing artifact**
short on time (15 min)	§1 (thesis + stack map), §5 (durability split), §8 (the monitor correction)	Figs. 1, 4, 8
a systems engineer	§4–§11 (substrate, single-writer, the theorems)	Thm. 11.1; Figs. 3, 11
a security reviewer	§8 (deontic split, the monitor), §13 (threat model)	Figs. 8, 7; Table 4
a protocol/agent author	§6 (claims), §7 (the bus), §14 (the interface contract)	Figs. 5, 6; §14
here for the economy	§9 (continuity organs), then the personhood and market papers	Fig. 10
an instructor	every **Exercises** block; the open problems OP-1 – OP-9 (§16)	the starred exercises

Reading order within the series. The legibility paper is the gentlest on-ramp; this paper, the substrate, is the formal floor; the personhood and market papers build upward. If you only read one paper to decide whether the foundation is sound, read this one.

3 The kernel as seven organs

We organize the kernel into seven *organs*, each a contract the daemon must hold for the layers above to be coherent (Figure 2). The temptation is to describe such a kernel as a flat noun-list of features — ports, claims, sessions, a bus, markers, commitments, a monitor, a memory store — which is the symptom of treating the kernel as a database. It is not a database. It is a system of record *plus* a mediator, and naming the organs surfaces the connective tissue a noun-list hides.

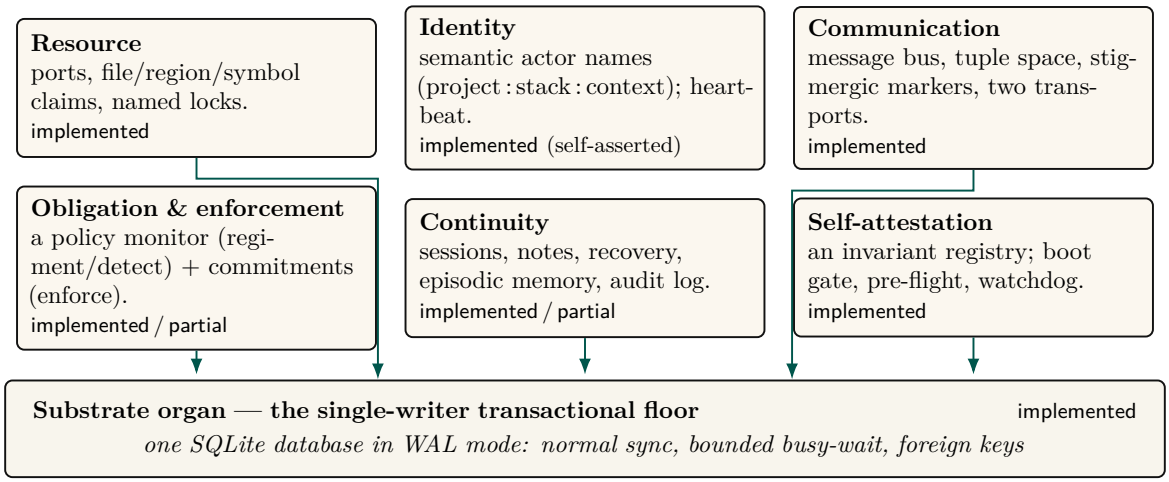

Figure 2: The kernel as *seven organs*, each a contract the daemon must hold for the layers above to be coherent. Six organs are, mechanically, just tables *with discipline* over a single shared file — the **substrate organ**, the single-writer transactional floor. The substrate's storage configuration *is* the kernel's durability claim (the I1a/I1b split, §11); a runtime-shim layer keeps the deployment and test SQLite bindings in step, and everything else is convenience built on top of it. Maturity grades (implemented · partial · specified · proposed) are per organ; note that the identity organ is implemented but *self-asserted*, the gap a cross-machine economy must eventually close with cryptographic identity.

The **substrate organ** is the floor; the other six are, mechanically, tables *with discipline* over the same file. We treat the substrate and the two organs that carry the paper's sharpest corrections (resource/exclusion and obligation/enforcement) in full, and the rest with the depth their novelty warrants.

Exercises.
(check) Which of the seven organs would still be meaningful if the daemon ran over an in-memory database with no disk at all? Which become vacuous?
(trace) Pick any one organ and name the single SQLite table that is its "home" table. Then name one invariant that organ owes the layer above it.
(open, ★) The seven-organ decomposition is a *choice*. Is there a more natural cut — e.g. five organs, or nine? Argue for an alternative and show what it makes easier to reason about and what it hides.

4 The substrate organ: one writer, one file

The substrate is the bedrock everything else is just a table in. Its job is to make two promises — durability and serialized mutation — and to make them formally. We cover the single-writer discipline here and durability in §5, because durability is where the most consequential correction lives.

4.1 The single-writer discipline

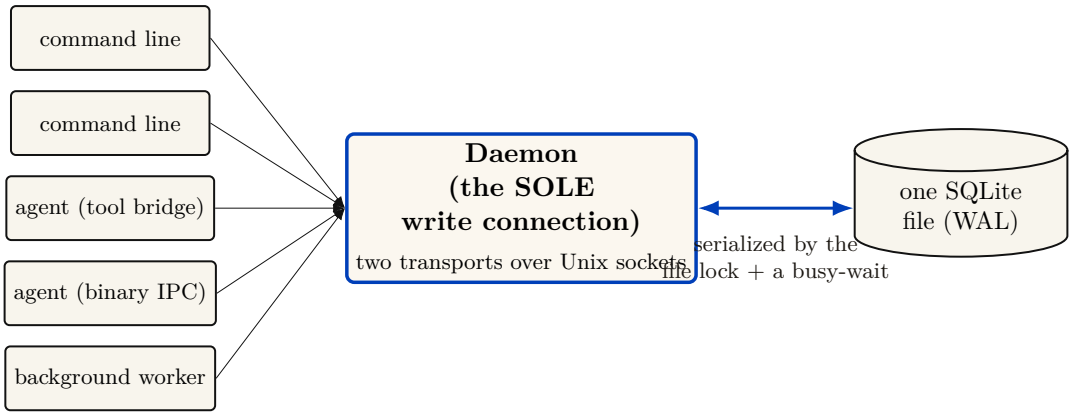

Figure 3: The single-writer serialization, the cleanest thing about the layer. Dozens of command-line invocations and agents funnel through *one* writer — the daemon, the sole holder of a read–write connection — and SQLite's OS-level file lock plus a bounded busy-wait serialize all mutations. Because there is exactly one decider, there is no agreement to reach: the consensus impossibility of Fischer–Lynch–Paterson does not bite — it is *sidestepped*, not solved, and on one machine the system is CP with a central coordinator. This is not a gap; it is the layer's defining discipline. The boundary is exact: the instant a *second* daemon shares this file, the assumption breaks and you have a distributed problem — which belongs to the cross-machine economy layer, not here.

The kernel's concurrency story (Figure 3) is: dozens of command-line invocations and agent connections funnel through one writer — the daemon, the sole holder of a read–write connection to the database file — and SQLite's operating-system file lock plus a bounded busy-wait serialize all mutations. We must be precise about *what* serializes writers, because there are two stories and only one is true at a time.

Definition 4.1 (Single-writer discipline)**.** All state-mutating operations route through a single daemon process holding one SQLite write connection. Concurrency among the many clients (command-line invocations, agents over the message transports, background workers) is resolved by the daemon's request queue; the SQLite file lock is the *backstop* that preserves correctness if the discipline is ever violated by a second writer (for example a stray direct write), not the primary serialization mechanism.

> **Key idea.** "Single-writer" is an architectural *discipline* (everything goes through the daemon), backstopped — not replaced — by SQLite's file lock. This distinction matters: SQLite permits multiple writer connections (it serializes them, it does not reject them), so the property holds because of where writes are routed, not because the database forbids a second writer. The standing risk is therefore a second *copy* of the daemon coming up against the same file — which is why the substrate also runs self-checks that detect a divergent or stale second instance (§10).

4.2 The configuration *is* the durability claim

The substrate's storage configuration is short and load-bearing. In WAL mode with the *normal* synchronization level (the WAL is fsync'd at checkpoints rather than on every commit), a bounded auto-checkpoint interval, a bounded busy-wait, foreign-key enforcement on, and a clean-shutdown checkpoint-and-truncate, the kernel inherits exactly the crash semantics that configuration defines. The claim "a write that returned success survives a crash" reduces *entirely* to what WAL with normal synchronization guarantees — which is the subject of §5, and which is not what an unguarded reading assumes.

4.3 Linear Migration Ledger (OP-7)

The initial schema-by-union approach (`CREATE TABLE IF NOT EXISTS`) created vulnerabilities around unsequenced renames and multi-column backfills. The kernel now enforces a strict, linear migration ledger managed exclusively by `agentsd` via SQLite's `pragma user_version`. Modules no longer self-initialize; instead, the daemon executes a strictly ordered set of migrations upon boot, guaranteeing a deterministic schema state for all subsequent agent access.

4.4 Path and ownership invariants

The database resolves to a single canonical path with owner-only file permissions (historically the file also carried the system's signing key in plaintext; that secret is being migrated to the operating system's keychain, see §13). A fail-closed guard refuses to open the live registry from a test context — the analogue of a framework's protected-environment check. This is Saltzer and Schroeder's *fail-safe defaults* [3] applied at the one place a test could corrupt production truth.

Exercises.

(check) The schema-by-union design lets any module create its tables in any order. Give a concrete two-module example where a lazy column-rename migration would be unsafe under this design.

(trace) Follow a single claim request from command line to disk. At how many points does the single-writer discipline (Def. 4.1) hold by *routing* versus by the *file lock*?

(open, ★ OP-7) Can idempotent self-init be extended to safe renames/backfills/down-migrations while preserving "any module, any order"? Or is a version table unavoidable?

5 Durability, split by fault class

This is the most important correction in the paper, so it gets its own section. The kernel's foundational promise — "a write that returned success survives" — is true, but only for the fault class a careful reader expects and not for the one the pitch most invites.

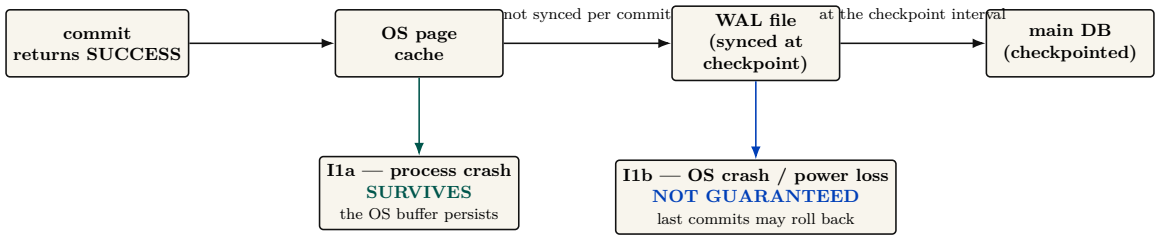

Figure 4: Durability by fault class — the single most important correction to the kernel's foundational promise, and exactly Gray & Reuter's atomicity-versus-durability distinction (the "D" in ACID is a separate promise). The unqualified claim "a write that returned success survives a crash" is true for a *process* crash (I1a: the OS page cache persists past the dying daemon) but *not guaranteed* for an OS crash or power loss (I1b), because under normal synchronization the WAL is synced only at checkpoints — the seductive belief that "normal sync is safe in WAL mode" conflates crash-*consistency* (true) with power-loss *durability* (false). The remedy for I1b is full synchronization or a checkpoint on the commit path; the kernel does not pay that cost by default, and the prose must not imply it does. We split the invariant accordingly: I1a is implemented; I1b is not guaranteed under the default WAL configuration.

5.1 Why one label is a lie here

Gray and Reuter [6] draw the canonical line between *atomicity/consistency* (a transaction is all-or-nothing and never leaves the store corrupt) and *durability* (the "D" in ACID: a committed transaction survives subsequent failures). Under SQLite's WAL with the *normal* synchronization level, the log is *not* fsync'd on every commit — it is synced at checkpoint [8]. Per SQLite's own documentation, in WAL mode the normal level means "commits might lose durability following a power loss or hard reset," though the database remains uncorrupted. So the honest statement splits in two.

Property 5.1 (Durability by fault class)**.** Let a write return success under the default WAL configuration.

- **I1a (process-crash durability).** If the *daemon process* dies (a forced kill, a panic, an out-of-memory termination), the write survives: the data is in the operating system's page cache, which outlives the process. **implemented**.
- **I1b (power-loss durability).** If the *operating system* crashes or power is lost *before the next checkpoint*, the most recent committed writes may roll back. *not guaranteed* under the normal synchronization level; it would require full synchronization or a checkpoint on the commit path.

Worked dramatization: Alice crashes mid-write. Alice is an agent editing a file. At t_0 she commits a claim on a port and a note recording her edit; the daemon returns success. At $t_0 + 5\,\text{ms}$ her host operating system kills the daemon process outright — a forced termination, no clean shutdown. *What survives?* Both writes. The committed pages live in the operating system's page cache, which the daemon process does not own and does not take with it when it dies; when the supervisor restarts the daemon, SQLite finds a dirty log, replays it, and Alice's claim and note are intact. This is I1a, and it is the fault class the design actually targets: agents die constantly, and the record must outlive them. Now change one detail. At $t_0 + 5\,\text{ms}$, instead of the daemon dying, *the power fails*. The committed pages were in the page cache but had not

yet reached stable storage, because under the normal synchronization level the log is fsync'd only at the next checkpoint. On reboot, the database is uncorrupted — but Alice's last claim and note may simply be gone, rolled back as if never written. This is I1b, and it is *not guaranteed*. The two faults differ only in whether the page cache survived; conflating them is the single most common over-claim about WAL-backed storage.

> **Pitfall.** The seductive misconception is "the normal sync level is safe in WAL mode because WAL already guarantees crash safety." This conflates crash-*consistency* (true: the database is never corrupt) with *durability* (false for power loss). The two are different ACID guarantees, and only the first is unconditional here. A systems paper cannot ship the unqualified claim. We split it.

5.2 Why the trade is defensible — and where it stops being so

Trading power-loss durability for throughput is a reasonable default for a local-first developer tool: the failure (lose the last few hundred milliseconds of claims after a power cut) is benign and self-correcting — agents re-claim, heartbeats resume. What is *not* defensible is letting the prose imply the stronger guarantee. To solve this (OP-10), the kernel implements **Selective Checkpointing**. High-stakes, money-bearing writes (like those in the settlement ledger) append `PRAGMA wal_checkpoint(FULL);` to their execution paths, forcing a synchronous flush to disk. This achieves power-loss durability (I1b) dynamically without compromising the high-throughput standard paths.

5.3 A recovery story, not just a durability story

Durability is about what survives; recovery is about what happens next. On daemon restart with a dirty WAL, SQLite replays the log to a consistent state (I1a's mechanism). But the kernel-level questions are larger: who validates the audit chain on boot, and what becomes of a half-applied cross-organ operation (§12) after a crash? The hook exists — the boot-admission gate that refuses to serve when a critical self-check fails (§10) — but boot-time WAL recovery plus integrity check plus chain verification should be a single named invariant, and today it is closer to PARTIAL than **implemented**.

Exercises.

(check) Classify each fault as I1a-survivable or I1b-exposed: (a) a forced kill of the daemon process; (b) a clean operating-system reboot; (c) yanking the power cord; (d) an unhandled exception in a request handler.

(trace) A claim commits at t_0; the next auto-checkpoint fires at $t_0 + \delta$. Draw the window during which a power loss reverts the claim, and state how a page-count auto-checkpoint threshold bounds δ.

(open, ★ OP-10) Design a per-write-path durability selector: how would the kernel let a settlement-ledger write opt into I1b while keeping claim writes fast? What is the cleanest interface that does not re-introduce the conflation?

6 The resource organ: claims, locks, and fair exclusion

Network ports are the namesake and the original sin: two agents that each start a development server on the same port collide, and one silently fails. The resource organ generalizes that problem into a single exclusion primitive — over network ports, over named mutex locks, and

over edit-surface claims on regions of source files — to which every higher layer's notion of ownership reduces.

6.1 Claim granularity is richer than "claims"

It is tempting to collapse all three of these into one word. The kernel instead distinguishes **whole-file**, **line-range**, and **symbol-path** claims, each keyed by the file path together with, respectively, nothing, a line interval, or a syntactic symbol identifier. This is the substrate for a "reserve regions, not whole files" coordination policy and for semantic-conflict prediction over a syntax-aware symbol index — but at the substrate layer it is simply exclusion at three granularities.

6.2 The claim lifecycle as a finite-state machine

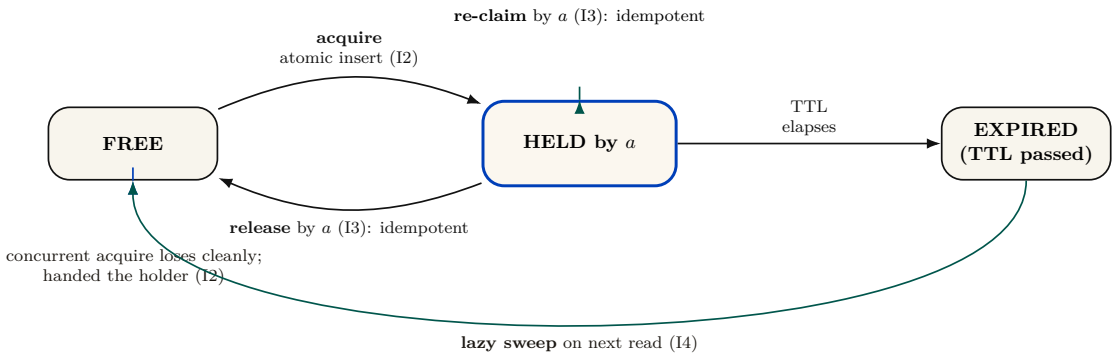

Figure 5: The claim/lock lifecycle as a finite-state machine, exhibiting the four properties on which the coordination layer builds typed ownership. *Acquire* is a single atomic insert against a uniqueness constraint (I2: a concurrent acquirer loses cleanly to a constraint violation and is handed the current holder). *Re-claim* and *release* are idempotent (I3): safe to retry, safe to double. Expiry is *lazy-swept* on the next read (I4) — expired rows are deleted when encountered, not on a timer. The conspicuous missing state is QUEUED: there is no wait-list and no fairness primitive, so a fast agent churning a hot resource can starve a slow one indefinitely — open problem OP-1, which asks whether bounded-wait can be added without dragging a scheduler down into the kernel.

Figure 5 states, as a state machine, the four properties on which the coordination layer builds typed ownership. The acquire path is small enough to state as an algorithm (Algorithm 1); we then give the central property as a theorem.

```
function acquire(key, requester, ttl):
  // single atomic statement; the uniqueness constraint is the
     mutex
  try:
    INSERT row (key, holder=requester, expires_at = now()+ttl)
    return GRANTED(holder=requester)          // we won
  catch UNIQUE_CONSTRAINT_VIOLATION:
    existing = SELECT row WHERE row.key = key
    if existing.holder == requester:
      UPDATE row SET expires_at = now()+ttl  // idempotent re-
         claim
      return GRANTED(holder=requester)
    if existing.expires_at < now():           // stale holder
      DELETE row WHERE key = key AND expires_at < now()
      return acquire(key, requester, ttl)     // one bounded
         retry
    return DENIED(holder=existing.holder)     // loser learns
       the winner
```

Listing 1: Claim acquisition. The whole acquire is a single insert against a uniqueness constraint; the loser is handed the current holder, not an error to retry blindly.

Theorem 6.1 (Atomic Mutual Exclusion and Fenced Leases)**.** Within a canonical conflict domain (exact normalized key, transactional interval overlap $[a, b] \cap [c, d] \neq \emptyset$, or path prefix hierarchy), at most one holder can hold an active lease at any logical instant. Furthermore, every granted lease carries a strictly monotonic fencing epoch $e \in \mathbb{N}$; downstream effect brokers reject any operation bearing an expired epoch $e' < e_{\text{current}}$.

Proof sketch. Within an immediate SQLite transaction (`BEGIN IMMEDIATE`), the daemon evaluates the conflict predicate against all currently unexpired leases. For exact keys, this is enforced by primary-key uniqueness; for intervals and hierarchies, by an indexed range check within the same transaction. Because all writes funnel through the single connection (Def. 4.1), predicate evaluation and insert are atomic. Upon grant or reallocation, the daemon increments the resource's fencing epoch e. Downstream effect brokers validate that e matches the active lease at execution time, preventing stale or paused holders from producing side effects after lease expiration. □ □

Worked dramatization: Alice and Bob race for one port. Alice and Bob are two agents, started a millisecond apart, that both want to bind a development server to port 7421. Each issues `acquire(7421)` (Algorithm 1). Both requests reach the single daemon, which serializes them: one insert lands first. Say Alice's does. Her insert succeeds and she is granted the port. Bob's insert, arriving an instant later, hits the uniqueness constraint on key 7421 and is rejected *atomically* — there is no moment at which both rows exist. Crucially, Bob does not get a bare error. The catch path reads back the row and hands Bob the *identity of the holder*: "port 7421 is held by Alice." Bob can now pick another port, wait, or message Alice, but he can never double-bind. The serialization that makes this clean is not a hand-rolled critical section; it is the operating system's file lock funnelling both inserts through one writer, and the database's primary-key uniqueness deciding the winner. One writer, one constraint, one holder.

Pitfall. Theorem 6.1 is sound for the *fresh-contention* case above. A subtler hazard hides in the *stale-holder* branch: reclaiming an expired lock runs a delete (of the expired row) and *then* an insert as two separate statements rather than one transaction. On the single daemon connection this is serialized by the connection itself and is safe. But if a second writer connection ever existed — the very scenario the single-writer discipline exists to prevent — a deferred transaction would open a window for a transient "database busy" error mid-sequence rather than clean serialization; the fix is to begin the transaction in immediate mode. So Theorem 6.1 holds because all writes route through the one daemon connection (Def. 4.1), *not* because the expire-then-acquire sequence is itself atomic. State which story you mean; do not tell both.

6.3 Fairness via Stigmergic Ticket-Lock (OP-1)

The conspicuous gap in the naive claim lifecycle is the absence of a QUEUED state. Locks have a time-to-live, but naive acquisition is racy first-come, meaning a fast agent churning a hot file can starve a slow agent indefinitely.

We solve this (closing open problem OP-1) via a **Stigmergic Ticket-Lock**. The substrate introduces a `claim_tickets` table:

```
CREATE TABLE claim_tickets (
  resource_id TEXT NOT NULL,
  agent_id TEXT NOT NULL,
  seq_num INTEGER PRIMARY KEY AUTOINCREMENT
);
```

When Agent B's claim acquisition fails against the primary-key `UNIQUE` constraint, the error handler inserts an entry into `claim_tickets`. When Agent A calls `release('auth.ts')`, the daemon executes a single atomic transaction:

1. Delete Agent A's active claim row: `DELETE FROM claims WHERE resource = :r;`
2. Read the lowest ticket: `SELECT agent_id, seq_num FROM claim_tickets WHERE resource_id = :r ORDER BY seq_num ASC LIMIT 1;`
3. Atomically re-grant the claim: `INSERT INTO claims (resource, holder, ttl) VALUES (:r, :next_agent, :ttl);`
4. Remove the claimed ticket: `DELETE FROM claim_tickets WHERE seq_num = :next_seq;`

Because the state transition is driven entirely *reactively* by the releasing agent in a single transaction, FIFO bounded-wait fairness is mathematically guaranteed by SQLite's B-Tree sequencing without dragging a background timer or polling thread into the kernel.

Exercises.
(check) Why does releasing a lock you do not hold return success rather than an error? Which property (I2–I4) is this, and what would break if it raised?
(trace) Two agents call `acquire` on the same fresh key at the same instant. Walk the uniqueness-constraint insert for both (Algorithm 1) and show exactly where the loser learns the holder's identity.
(solution, ★ OP-1) The Stigmergic Ticket-Lock provides bounded-wait fairness purely through the reactive release transaction without requiring a kernel background scheduler.

7 The communication organ: the bus, stigmergy, and two delegation chains

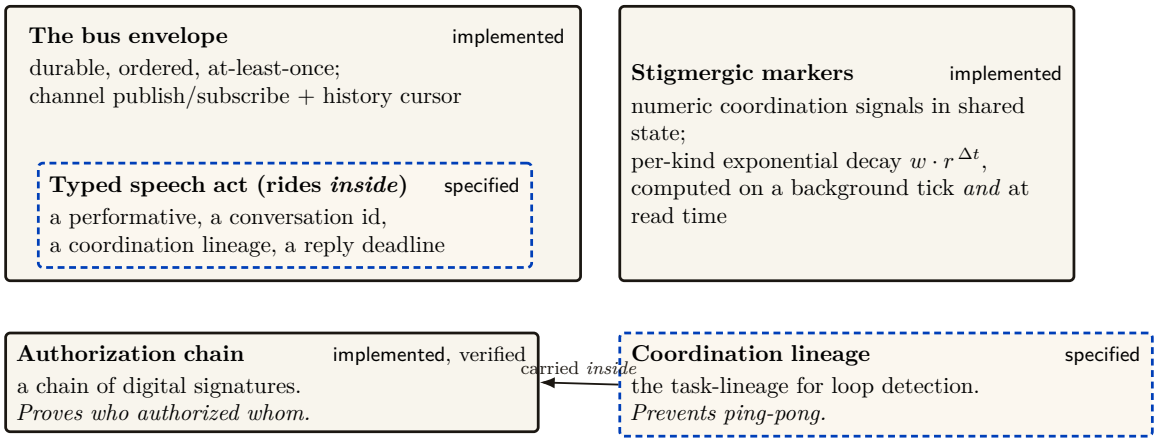

Figure 6: The communication organ. The **bus** envelope — durable, ordered, at-least-once, history-cursored (implemented) — is the carrier *into which* the typed speech act is layered; the kernel ships the envelope, the coordination layer ships the semantics that ride inside it. **Stigmergic markers** implement coordination with per-kind exponential decay computed both on a background tick and *at read time*, so the anti-blackboard-rot property is provided by the substrate rather than left to discipline. Finally, the figure fixes a dangerous naming collision — never write bare "delegation chain," it is two objects: the *authorization chain* is the hop-bound, attenuation-monotonic, anti-replay chain of signatures that proves *who authorized whom*; the *coordination lineage* is the coordination object that prevents ping-pong — a different thing entirely, carried *inside* the authorization chain.

The communication organ (Figure 6) is where the substrate stops being a lockbox and becomes a medium for conversation. The carrier is **implemented**; the *semantics* that ride on top of it (typed speech acts, protocol state machines) belong to the coordination layer. Three pieces matter here.

7.1 The bus: a durable carrier envelope

The bus is a durable, ordered, *at-least-once* publish/subscribe channel over a messages table, with a versioned envelope (a version tag, a message kind, a body, and an in-reply-to pointer), a history cursor for replay, and a per-message time-to-live. The coordination layer's typed speech act — carrying a performative, a conversation identifier, a coordination lineage, and a reply deadline — is layered *inside* this envelope. The kernel ships the carrier; the coordination layer ships what it carries.

> **Key idea.** At-least-once delivery means a *healthy* bus routinely redelivers and reorders. This collides with a loud-fail honesty discipline: if "every anomaly is loud," benign redelivery drowns the operator in false alarms. The resolution is that benign transport non-determinism must be *deduplicated silently*, and only genuine protocol-state-machine violations surface loudly. That requires an idempotency key in the envelope — currently absent — which is the cheapest high-leverage fix at the coordination layer. The kernel provides the bus; making every speech-act effect idempotent is the coordination layer's debt, flagged here so the reader does not mistake silent deduplication for dishonesty.

7.2 Stigmergic markers: decay as a provided guarantee

The kernel also carries *stigmergic markers* — numeric coordination signals deposited in shared state, after the manner of Grassé's stigmergy in social insects [16] and the generative-communication tuple spaces of Gelernter's Linda [17], and used in ant-colony optimization [18]. Each marker carries per-kind exponential decay $w \cdot r^{\Delta t}$, computed both on a background evaporation tick *and* at read time (so a reader sees the correctly decayed value without waiting for the tick), and pruned below a small threshold. The decay is the structural defense against "blackboard rot": stale coordination signals evaporate on their own. The calibration of the decay rate is genuinely open (OP-8): too slow and signals rot, too fast and they evaporate before they coordinate.

7.3 Two delegation chains, one dangerous name

A dangerous naming collision lives in this organ. The phrase "delegation chain" names *two different objects*, which we keep rigorously distinct:

- the **authorization chain** — the *cryptographic* object (**implemented**, mechanically verified in a protocol-verification tool): a hop-bound, attenuation-monotonic, anti-replay and anti-splice chain of digital signatures that proves *who authorized whom*. This is what cross-machine capability transfer in the economy layer rides on.
- the **coordination lineage** — the *coordination* object (SPECIFIED): the task-lineage over which loop detection and upward-block run, preventing two agents from delegating a task back and forth forever.

These are distinct objects, and the relationship is that the coordination lineage is carried *inside* the authorization chain — loop detection runs over a lineage whose authenticity the cryptographic chain guarantees. We never again write bare "delegation chain."

> **Pitfall.** The "lineage inside chain" relationship is a slogan until the task-shape field is in the *signed* payload at each hop — and that collides with a rule against keyword-based text classification, because loop detection needs a *semantic* task-shape equivalence (a learned embedding or a structural hash), which is not deterministically signable. This tension is a real open problem; we flag it rather than assert it solved.

7.4 A second transport

The kernel ships *two* transports, not one: a request/response protocol over a Unix domain socket *and* a binary framed transport over a Unix domain socket, the latter with peer-credential authentication and backpressure/draining for high-frequency, tight-loop coordination. We note in §13 that the peer-credential check is a software handshake, not the kernel-enforced socket-level credential check (`SO_PEERCRED`) it resembles — a real trust-boundary caveat.

Exercises.

(check) Why does read-time marker decay make the system more honest than a background tick alone? Give a scenario where tick-only decay would report a stale signal as fresh.

(trace) A request speech act is sent over the bus and redelivered once by the at-least-once channel. Trace what the operator should see under the silent-dedup resolution, and what an envelope idempotency key would change.

(open, ★ OP-8) What per-kind marker decay rate keeps stigmergic signals neither rotten nor prematurely evaporated? Frame this as an empirical measurement, not a constant to guess.

8 The obligation & enforcement organ: the sovereign's two arms

This organ is the deontic core, and it is genuinely two distinct mechanisms — in the vocabulary of deontic logic for computer systems (Jones and Sergot [5]): *prohibition* (a policy monitor) and *obligation* (commitments). Getting the distinction right — and being honest about how strong each arm actually is — is the security heart of the paper.

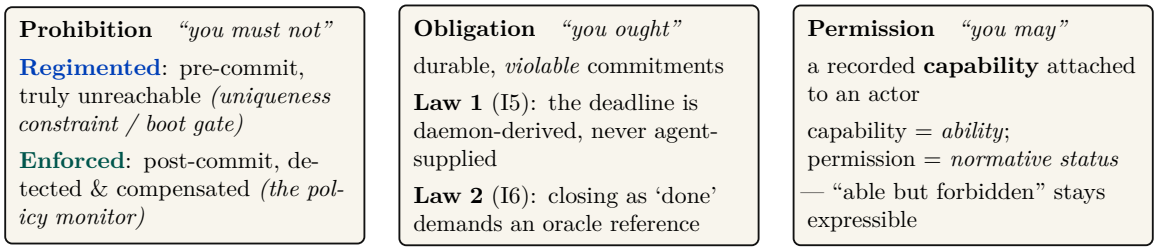

Figure 7: The kernel's deontic split (Jones & Sergot), the formal core of its obligation/enforcement organ: *prohibition* is regimented *or* detected, *obligation* is enforced by oracle-bound commitments, *permission* is a recorded capability — and the coordination layer's deontic binding is a direct lift of this split. The regimented set (double-claim, duplicate-identity squat, serve-from-test-database, refuse-to-serve on a failed self-check) is rejected pre-commit by a uniqueness constraint or fail-closed boot gate; everything the policy monitor *subscribes* to (heartbeat freshness, note monotonicity, capability escalation, lock-owner validity) is detected post-commit and compensated. Commitments follow Cohen & Levesque's persistent goals: open → done | abandoned | superseded, with Law 1 closing the Goodhart hole on deadlines and Law 2 refusing free-text closure. And capability (ability) is kept distinct from permission (normative status) so that "able but forbidden" — e.g. a dangerous override flag that *exists* but *must not be used* — remains representable, the very state a guardrail-bypass discipline depends on.

8.1 The deontic split

Definition 8.1 (The kernel's deontic split)**.** The kernel realizes three deontic modalities with three mechanisms: *prohibition* is regimented (rejected pre-commit) or enforced (detected post-commit); *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability. The coordination layer's deontic binding is a direct lift of this split.

We keep **capability** (*ability* — what the agent's sandbox makes possible) distinct from **permission** (*normative status* — what the deontic layer allows), so that "able but forbidden" stays expressible. The canonical example is a dangerous override flag that a tool exposes for emergencies: it *exists* (an agent is *capable* of invoking it) but *must not be used* (it is *forbidden*). Collapsing capability into permission makes that state inexpressible — which is exactly the bug a discipline against silently bypassing guardrails cannot afford.

8.2 The policy monitor is a detector, not a regimenter

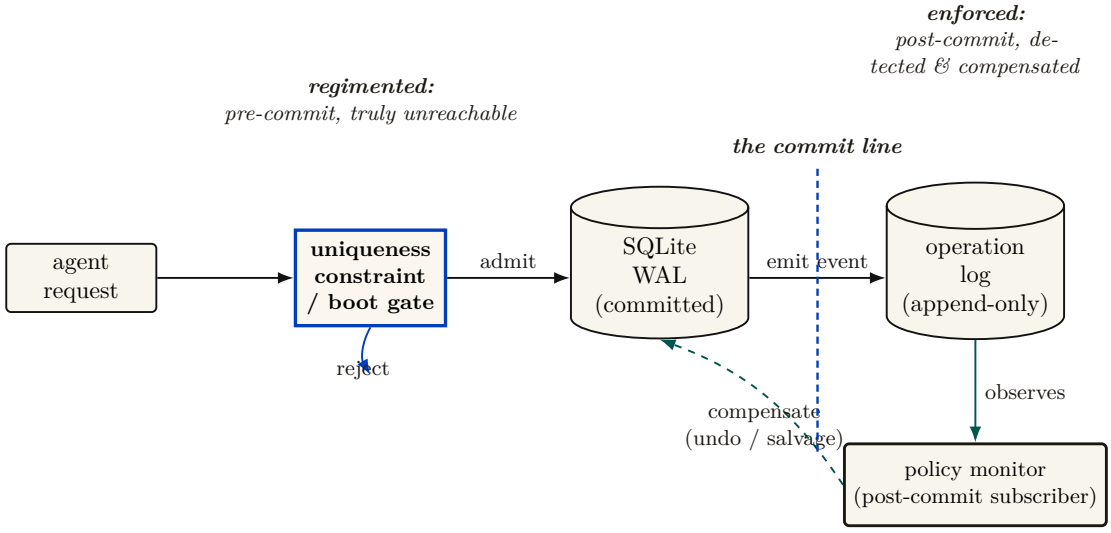

Figure 8: The kernel as a reference monitor — and the honest correction it forces. Anderson's reference monitor must be *always invoked* on the request path (complete mediation), *tamper-evident*, and *small enough to verify*. The **uniqueness constraint / boot gate** (cobalt) meets the first property by construction — the single SQLite file lock means every double-claim, duplicate-identity squat, or serve-from-test-database attempt is rejected *before* the WAL write. The **policy monitor** (teal) does not: it is a subscriber to the operation log that fires *after* the event is already durable, and by Schneider's theorem [4] (§8) a monitor downstream of the commit line enforces no safety property — it can only *detect and compensate* within a bounded window. Tamper-evidence (a hash-chained audit log) is re-scoped to the economy layer (§13.4); smallness is the point of one writer over one file. We therefore reserve "physically unreachable" for the regimented set and say "detected, then halted or compensated" everywhere else.

Here is the correction that the rest of this stack must adopt. The most seductive claim one can make about a coordination policy monitor — that it "makes forbidden states physically unreachable" — is false as stated. The monitor is a *subscriber to an append-only event stream of operations that have already committed*. By the time it observes a violation, the offending write is already durable in the log. Even in its strictest mode, its strongest response is a *compensating* undo, not a prevention. This is not an implementation accident; it is a theorem.

Theorem 8.1 (A prevention bound for post-commit monitoring)**.** A monitor invoked only after a transition commits cannot prevent a safety violation whose bad prefix ends at that transition. It can detect the prefix, halt later actions, and enforce a separate recovery or bounded-response property. Therefore a post-commit subscription must not be credited with making the triggering state unreachable.

Proof sketch. Schneider's execution-monitor model [4] recognizes safety properties through finite bad prefixes and enforces them by suppressing an action before that prefix enters the target execution. A subscriber invoked after commit cannot suppress the committing action. Compensation may establish a different eventual or bounded-recovery property, but it does not retroactively remove the bad prefix. □ □

Worked dramatization: Bob attempts a forbidden action. Bob is an agent under a policy that forbids editing files outside his assigned task scope. He nonetheless issues a write that records an out-of-scope edit. *What happens?* The write commits. The single writer serializes it, the uniqueness constraint has no objection (it is a well-formed row), and the operation lands durably in the log. Only *then* does the policy monitor, subscribed to that log, observe the out-of-scope edit and raise a violation. Its response is compensating: it can flag Bob's session,

revoke his claims, alert the operator, and trigger a salvage of the affected surface — but it cannot pretend the write never happened, because it did. Contrast this with Bob trying to claim a port Alice already holds (the previous dramatization): *that* attempt never commits, because the uniqueness constraint rejects it before the commit line. The two cases are the whole deontic story in miniature: a uniqueness constraint and a boot-time admission gate *regiment* (the bad state is unreachable); the policy monitor *enforces* (the bad state is reached, then detected and compensated within a bounded window).

> **Key idea.** The only genuine pre-commit regimentation is credited to the right mechanism: the uniqueness constraint (no two holders of one resource key; no duplicate process-identity squat) and the fail-closed boot-admission gate (no serving from a test database; refuse to serve when a critical self-check fails). Those reject *before* the commit line and are *truly* unreachable. The policy monitor notices everything else *afterward* and compensates. So: "physically unreachable" for the regimented set; "detected within a bounded window, then halted or compensated" for the rest. Crediting double-claim prevention to the monitor *double-counts* — the uniqueness constraint already did it; the monitor merely logs it.

Theorem 8.2 (Synchronous State Decidability (OP-2))**.** A safety invariant ϕ is **regimentable** (strictly preventable pre-commit, making violation states physically unreachable) if and only if $\phi(S, \Delta)$ is a pure, deterministic boolean predicate decidable solely over the daemon's local, synchronous, committed database state S and the incoming transaction payload Δ.

Conversely, any invariant requiring external I/O, asynchronous wall-clock verification, distributed consensus, or non-deterministic grading oracles is strictly **enforceable** (post-commit, detect-and-compensate).

Proof sketch. In SQLite WAL mode under single-writer serialization, transaction commits occur synchronously on the database connection thread. A predicate $\phi(S, \Delta)$ computable in bounded CPU steps without yielding the event loop can be embedded directly into SQLite constraints (`UNIQUE`, `CHECK`) or transaction guard clauses; any transition yielding $\neg\phi$ rolls back atomically before state mutation. If ϕ depends on external network I/O or asynchronous delays, holding the exclusive transaction lock during evaluation starves the single-writer queue; hence evaluation must be deferred post-commit, converting the mechanism from a synchronous gate into an asynchronous event subscriber governed by Theorem 8.1. □ □

Remark (Scope: the general boundary is controllability). Theorem 8.2 is the *operational* form of a more general boundary. In supervisory-control terms (Ramadge–Wonham, 1987), partition the event alphabet into controllable events Σ_c (mediated effects the daemon can refuse pre-effect: writes, egress, tool execution, spawns) and uncontrollable events Σ_u (model-internal steps: token emission, in-context reads, plan formation). A safety policy is regimentable iff it is *controllable* with respect to Σ_u — no uncontrollable event enabled after a legal prefix can exit the specification. Theorem 8.2 is the special case $\Sigma_c = \{\text{synchronous DB commits}\}$. Two consequences the special case obscures: (i) semantic properties of model output ("no confident falsehood in a report") forbid an *uncontrollable* event and are therefore detect-only *forever*, at any engineering budget; (ii) compound policies that *permit* an uncontrollable trigger while gating its controllable consequence — "no egress after reading a secret" — *are* regimentable: record the read as taint, refuse the egress. Gate the channel, never the token.

The monitor is honest about its own coverage, which is itself a genuine security property: it reports each rule as *enforced*, *degraded*, or *stubbed*, so a reader can never mistake a stubbed rule for an active one. We note in §13 that one rule's enforcement (capability escalation) is backed by a separate native component whose evidence is the weakest in the organ, carrying a time-of-check-to-time-of-use surface.

8.3 Commitments: obligation with oracle-bound closure

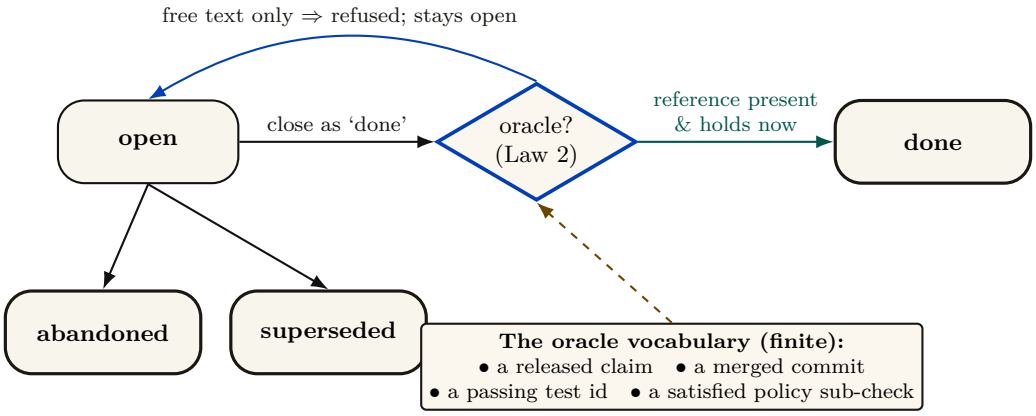

Figure 9: The commitment lifecycle with its oracle-bound closure gate. A commitment is a durable, *violable* promise (Cohen & Levesque's persistent goal): it may end *done*, *abandoned*, or *superseded*. The load-bearing gate is Law 2 (I6): a transition to *done* is *refused* — the promise stays open — unless the agent supplies a reference drawn from the finite oracle vocabulary, re-verified by the daemon at closure time; free-text notes cannot close a promise. Law 1 (I5) closes the complementary Goodhart hole: the deadline is derived by the daemon from a scope policy — the agent picks the *work*, never the *deadline*. That finite vocabulary is precisely the kernel's honesty boundary (OP-5): it can mechanically verify "the test you named passed," never "the refactor is good."

Commitments are durable, *violable* promises — in the sense of Cohen and Levesque's "intention as a persistent goal" [14], with both single-minded and open-minded strategies for when a promise may be dropped. Two hardening laws are implemented, and they are the load-bearing ones (Figure 9).

Property 8.1 (Metric-control separation, Law 1)**.** A commitment's deadline is derived by the daemon from a scope policy and is *never* agent-supplied. The agent picks the work; the daemon picks the deadline. (Invariant I5.) The point is to deny the agent control of the very metric it is judged against. This removes one direct manipulation channel; it is not a general proof of resistance to proxy gaming or misspecified scope policy.

Property 8.2 (Oracle-bound closure, Law 2)**.** Closing a commitment as "done" is refused unless the agent supplies a non-empty reference drawn from a *finite oracle vocabulary*: a released claim, a merged commit hash, a passing test identifier, or a satisfied policy sub-check. Free-text notes cannot close a commitment. (Invariant I6.)

The closure gate is small enough to state as a protocol (Algorithm 2). The crux is the final clause: the supplied reference must not only be drawn from the vocabulary but must *currently hold* when the daemon re-checks it, so an agent cannot name a test that passed yesterday and has since regressed.

```
function close(commitment, status, oracle_ref):
  if status != 'done':
    return apply_drop_strategy(commitment, status)   // abandon / supersede
  if oracle_ref is empty:
    return REFUSED("done requires a verifiable oracle reference")
  kind = classify(oracle_ref)                        // structured field, not NLP
  if kind not in {RELEASED_CLAIM, MERGED_COMMIT, PASSING_TEST, POLICY_SUBCHECK}:
    return REFUSED("oracle reference outside the finite vocabulary")
  if not verify_holds_now(kind, oracle_ref):         // re-check at closure time
    return REFUSED("named oracle does not currently hold")
  transition(commitment, DONE, witnessed_by = oracle_ref)
  return CLOSED
```

Listing 2: The commitment-closure oracle. Closure requires a reference from a finite vocabulary that the daemon re-verifies at closure time; free text is rejected by construction, not by inspection.

> **Key idea.** That finite oracle vocabulary *is* the kernel's honesty boundary. It can mechanically verify "the test you named passed," never "the refactor is good." Naming the oracle set is naming the limit of what the substrate layer can verify — and open problem OP-5 asks which real "done" conditions it fails to capture, and whether the set can be extended without re-opening the Goodhart hole that free-text closure would.

8.4 Formalizing Synchronous State Decidability (OP-2)

The boundary between regimentation (pre-commit rejection) and enforcement (post-commit detection) is formally determined by **Synchronous State Decidability**. A monitor rule is *regimentable* if and only its validity can be decided purely from the deterministically available local database state and the incoming payload, without any async yield, external I/O, or clock non-determinism. If evaluating the rule requires a network call (e.g., verifying an external API token) or a non-deterministic oracle (e.g., waiting for an LLM response), it crosses the decidability boundary and must be downgraded to *enforceable* (detected asynchronously by a background sweep).

8.5 Cryptographic State-Machine Assertions (CSMA) (OP-5)

To capture real completion conditions without re-admitting the Goodhart hole of free-text claims, we introduce **Cryptographic State-Machine Assertions (CSMA)**. CSMA extends the oracle vocabulary by adding *Delta Oracles* (which prove that pre/post benchmark execution timing improved by a threshold) and *AST Assertions* (which use `tree-sitter` to structurally validate code modifications). By restricting "done" conditions to these cryptographically verifiable state transitions, the kernel achieves oracle completeness without sacrificing mechanical verification.

Exercises.

(check) For each monitor rule in Theorem 8.1, say whether it *could in principle* be moved to a pre-commit check (a before-insert trigger). Which genuinely cannot, and why? (Hint: heartbeat freshness is a failure detector, and Chandra and Toueg [12] show failure detection cannot be made perfectly synchronous.)

(trace) An agent attempts to close a commitment with an empty oracle reference. Walk the Law-2 gate and show exactly where and why the transition is refused.

(open, ★ OP-2) Formalize the regimentation-vs-enforcement boundary: which monitor rules are truly regimentable (rejectable at insert) versus only enforceable (detectable after)? A clean theorem here is the deontic heart of the layer.

(open, ★ OP-5) Extend the Law-2 oracle vocabulary to capture a "done" condition it currently misses, without re-admitting free text. State your new oracle and prove it is Goodhart-resistant.

9 The continuity organ: memory, checkpoint, and the foundation of the economy

The through-line of the whole series — memory → checkpoint → continuity → durable identity → reputation → market — *bottoms out here.* If the substrate's continuity is weak, the cross-machine economy is built on sand. This is the section the personhood and market papers lean on hardest, so it carries the canonical statement of the layer's own softest link.

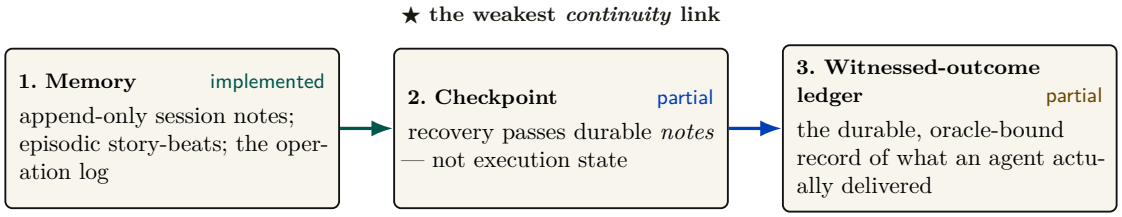

Figure 10: The three continuity organs the entire economy stands on — the canonical sentence the rest of the series depends on. The through-line of the series — memory → checkpoint → continuity → durable identity → reputation → market — bottoms out *here*, in the kernel. **Memory** is implemented, and the kernel even forbids amnesia (a monitor rule keeps an active session's note count from regressing). **Checkpoint** is partial: recovery (heartbeat → stale/dead → queue → salvage handoff) passes the durable notes but does not capture execution state, so it is the weakest continuity link — a checkpoint with teeth (a real execution-state snapshot, OP-4) is the literal foundation of a cross-machine reputation economy. **The witnessed-outcome ledger**, on which reputation actually keys, has a partial shipped substrate: durable commitments, daemon-derived deadlines, oracle-bound closure, and overdue detection. Neutral graded outcomes, sanctions, and reputation binding do not yet exist. One "the foundation is soft here" claim, not three.

9.1 Three organs, one weak link

Figure 10 names the three continuity organs and states the canonical sentence the rest of the series depends on:

> *The economy rests on three continuity organs — memory (IMPLEMENTED), checkpoint (PARTIAL: notes, not execution state), and a witnessed-outcome ledger (PARTIAL: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third organ, which does not yet exist; the checkpoint organ is the weakest continuity link, and a checkpoint with teeth (a real execution-state snapshot) is the literal foundation of the cross-machine economy.*

Memory — session records, durable notes, and the operation log — is **implemented**, and the kernel even forbids amnesia: a monitor rule guards that an active session's durable note count never regresses (detect-and-compensate, per Theorem 8.1, but real).

Checkpoint is PARTIAL, and the weakness is precise. The recovery pipeline runs a heartbeat watchdog → stale/dead detection → a recovery queue → a salvage handoff, and it *passes notes*; it does not checkpoint execution state. A checkpoint with teeth — the gap between passing notes and restoring work — is open problem OP-4, the most important unbuilt thing at this layer because it is the literal foundation of any reputation economy.

The witnessed-outcome ledger — the durable record of what an agent actually delivered — is the organ on which reputation keys. A PARTIAL substrate now ships: durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI surfaces, and overdue detection. Neutral graded outcome events, sanctions, identity binding, and reputation updates remain absent. OP-4 (here) and that richer outcome-ledger gap (the personhood paper) are the *same* finding viewed from two ends.

9.2 The operation log and tamper-evidence

The append-only operation log is the kernel's audit organ — the stream the policy monitor subscribes to and the thing that makes "what actually happened" answerable. Hash-chained tamper-evidence over it — the digital-timestamping pattern of Haber and Stornetta [20] that predates blockchains, itself building on Merkle's hash trees [19] — exists but is re-scoped (see §13.4).

Exercises.

(check) Why is "recovery passes notes" fundamentally weaker than "recovery restores work" for a language-model agent that died mid-edit? Name one thing notes preserve and one thing they cannot.

(trace) Follow the note-monotonicity rule from an agent appending a note to the monitor detecting a regression. At which point is the violation already durable (per Theorem 8.1)?

(open, ★ OP-4) What is the minimal durable artifact that turns "recovery passes notes" into "recovery restores work"? Is it a content-addressed snapshot of {working-tree diff + open claims + commitment set + the last N turns of the agent's transcript}? What is recoverable versus fundamentally lost when a language-model agent dies mid-thought?

10 The self-attestation organ: honest green

Honest self-attestation belongs to the substrate layer, because the kernel is the one component that can formally report its own integrity: it is always-on and owns the only writable truth. The attestation routine evaluates a registry of invariants, each returning *pass*, *fail*, *skipped-with-reason*,

or *unknown*. The report is *scoped and conjunctive*: "all good" is printed only when every checked invariant passes, and the report always enumerates what was *not* checkable.

Property 10.1 (Honest self-report, I10)**.** The attestation reports "all good" only if every checked critical invariant passed, and it always lists the unchecked set. There are three triggers: a boot-admission gate (refuse to serve on a critical failure), a command pre-flight (loud refusal that names the fix), and a continuous watchdog (a pass-to-fail transition deposits a coordination marker and a notification). **implemented**.

This is the honesty discipline of the rest of the series turned on the kernel itself, and it is the right posture for a trusted computing base: when integrity is unknown, deny. The drift and liveness self-checks exist precisely because a second copy of the daemon *can* come up against the same file — a packaged install lagging the development tree is a recurring hazard — and the kernel must know which copy of itself is the real one.

11 The invariants, stated as theorems

A completionist treatment names the kernel's properties as falsifiable claims, each with its mechanism and its maturity grade. Table 3 is the formal spine of the layer; the grade column is the maturity discipline turned on the theorems themselves.

Table 3: The kernel invariants. I1 is split by fault class (I1a/I1b) per §5; I7–I9 carry the regimentation/detection correction per §8.

#	Invariant (theorem)	Mechanism	Maturity
I1a	Process-crash durability	WAL + OS page cache	**implemented**
I1b	Power-loss durability	(needs full synchronization)	*not guaranteed*
I2	Mutual exclusion (1 holder/key)	uniqueness constraint + busy-wait	**implemented**
I3	Idempotence of re-claim/re-release	upsert / delete-if-exists	**implemented**
I4	Liveness reclamation (TTL sweep)	lazy expiry on read + ticks	**implemented**
I5	Metric-control separation (Law 1)	daemon-derived deadline	**implemented**
I6	Oracle-bound closure (Law 2)	finite oracle vocabulary	**implemented**
I7	No-amnesia (note monotonicity)	monitor: note-monotonicity rule	PARTIAL (detected, not regimented)
I8	Forbidden-state handling	constraint/boot (regimented) + monitor (detected)	PARTIAL (mostly detect-and-compensate)
I9	Tamper-evidence of the audit log	hash chain	PARTIAL (re-scoped to the economy layer, §13.4)
I10	Honest self-report	scoped attestation report	**implemented**
I11	Runtime parity (interpreter ≡ test bindings)	runtime-shim layer	PARTIAL (OP-3)
I12	Non-forgeable identity	actor-soul id + lookup credential	PARTIAL (bounded gate ships; universal write gating absent)

11.1 The consistency-model theorem

The strongest words a single-writer design earns are "serializable" and "linearizable." They are the formal payoff of the architecture, and they are what make the coordination layer's typed ownership trustworthy.

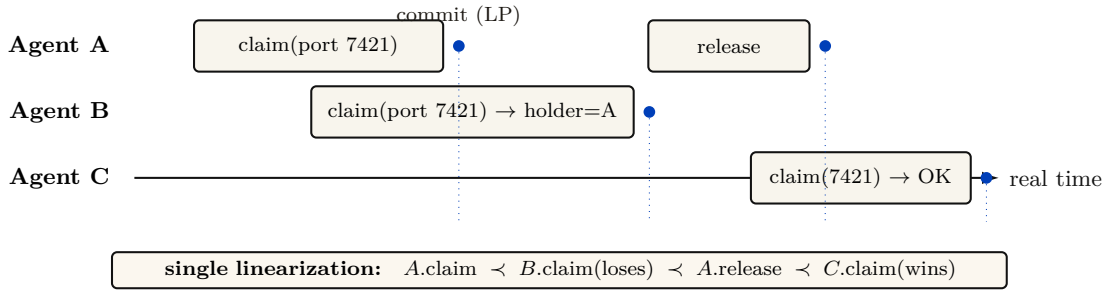

Figure 11: The consistency-model theorem — the formal property the single-writer choice buys, made visible. Because all mutations serialize through one writer over one WAL file, transactions are **serializable**; because that writer is the sole decider, the observable history of claims and locks is **linearizable**: every operation takes effect atomically at its commit (its *linearization point*, the cobalt dots), and the commit order coincides with real time. The trace shows A winning a port, B losing cleanly to the holder, A releasing, and C then winning — one unambiguous order with no agreement protocol in sight. This linearizability is exactly what lets the coordination layer treat "who holds this" as ground truth.

Theorem 11.1 (Conditional consistency model)**.** Assume (i) every claim/lock read and write goes through one daemon and one SQLite database connection, (ii) transactions do not enable `read_uncommitted`, (iii) each successful response is sent only after commit, and (iv) no out-of-band writer mutates the tables. Then committed transactions admit a serial order, and the externally observed claim/lock operations are linearizable with commit as the linearization point.

Proof sketch. Under the stated configuration there is one stream of write transactions, totally ordered by commit, and reads observe a consistent snapshot. For linearizability (Herlihy & Wing [10]) we need a single point per operation, between its invocation and response, at which it appears to take effect, with these points consistent with real time. The commit of each claim/lock operation is exactly such a point: it is atomic (Theorem 6.1), it lies between the client's request and the daemon's response. Non-overlapping operations respect real-time order because a later invocation occurs after the earlier response and therefore after its commit; overlapping operations may be ordered by commit. Hence the observable history is linearizable. □ □

> **Key idea.** This is why the kernel needs no agreement protocol inside one fault domain. Linearizability follows from the routing, transaction, and response-order conditions above; it is lost if a second writer or an out-of-band read bypasses those conditions. One decider, one order, one auditable truth.

11.2 The dual-runtime hazard, and what would make I11 a theorem

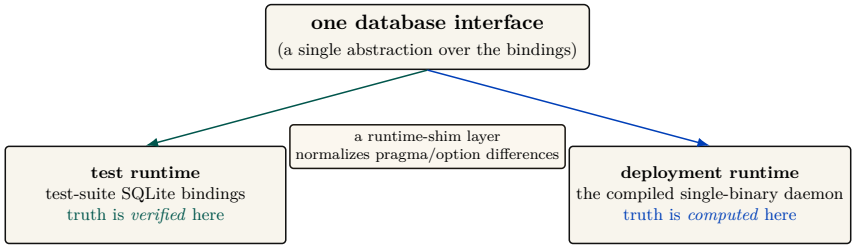

Figure 12: The dual-runtime substrate, a genuine kernel property easily mistaken for an implementation detail. One database interface is satisfied by *two* sets of SQLite bindings: the test-suite bindings (where invariants are *verified*) and the deployment bindings in the compiled daemon (where truth is actually *computed*). A shim normalizes their pragma and option interfaces, but the bridge is not a proof: an invariant green under test can fail in deployment if the bindings diverge in lock or pragma semantics — the recurring "green in the test runtime, broken in deployment" failure, which is why CI must *boot the deployment runtime* and exercise its endpoints, not merely build it. Invariant I11 (runtime parity) is therefore partial, and open problem OP-3 asks for the differential-test harness — identical operation sequences against both runtimes, asserting identical observable state — that would make it a theorem.

Invariant I11 (runtime parity) is the one place the formal spine is held together by a compatibility shim rather than a proof. In deployment, truth is *computed* under the SQLite bindings of one runtime (the compiled single-binary daemon), but in the test suite it is *verified* under a different runtime's bindings; a thin shim normalizes their pragma and option interfaces (Figure 12). An invariant that is green under test can fail in production if the two bindings diverge in lock or pragma semantics — the classic "green in the test runtime, broken in the deployment runtime" failure. A continuous-integration pipeline must therefore *boot the deployment runtime* and exercise its endpoints, not merely build it. To solve this (formerly open problem OP-3), the kernel's CI pipeline runs **Differential Fuzzing**: 1×10^6 concurrent operations are blasted against both the test and deployment SQLite bindings, asserting strict hash equivalence of the resulting database state. This guarantees runtime parity across bindings.

Exercises.
(check) List the four assumptions Theorem 11.1 charges for linearizability. Which one fails first if an out-of-band SQLite connection is introduced?
(trace) Construct a three-operation claim history and give a valid linearization. If two calls overlap, show why more than one linearization may still respect real time.
(open, ★ OP-3) Specify the minimal differential-test suite that would make I11 a theorem: what operation sequences, what observable-state assertions, run against both runtimes?

12 Cross-organ atomicity: a buildable defect, not a frontier

"Claim this port *and* open this session *and* record this commitment" is three writes. Nothing at the kernel's interface boundary wraps them in one transaction, so partial-failure semantics across organs are undefined. It is tempting to file this under "research"; that is wrong, and the correction matters because it changes the engineering posture from "research" to "do it."

> **Key idea.** This is the Saga problem (Garcia-Molina & Salem [15]) — but with a *cheap local fix*. These are all local writes to one file, and the database already provides a transaction primitive (used elsewhere in the system). Wrapping a multi-organ operation in one local transaction (begun in immediate mode) gives all-or-nothing for free. Filing this under "research" understates how cheap the correct answer is. It is a **buildable defect**: the durable primitive is present; what is missing is the wrapping of the multi-organ endpoints in it.

This is the kind of finding the maturity discipline exists to surface: a gap mislabeled as a research frontier hides a one-line fix behind a grander word. Sagas matter when the steps are remote and a single transaction is impossible; here, where every step is a local write to one file, the single transaction is strictly simpler and strictly stronger.

Exercises.

(check) Give a concrete partial-failure: port claimed, session opened, then the commitment write fails. What inconsistent state is left, and which agent sees it?

(open, ★ OP-11) Sketch the interface change that wraps the three-organ "begin" operation in one local transaction. What is the compensating action if you instead chose Sagas [15], and why is the transaction strictly simpler here?

13 Threat model and the limits of mediation

A systems-security paper lives or dies on its threat model. The kernel deliberately shrinks its trusted computing base (one writer, one file), which is good security hygiene — but the shrinking pushes several adversaries explicitly out of scope, and the honest move is to draw the boundary, not gesture at it.

Table 4: The substrate-layer threat model. The same-user adversary is *out of scope* by design; several "security" organs only become load-bearing against the cross-machine adversary, which belongs to the economy layer.

Adversary	In/out of scope	What defends (or does not)
Buggy cooperating agent	**in**	Claims, commitments, post-commit monitoring, oracle-bound closure. The primary design target.
Misbehaving agent (ignores advisory claims)	partial	Edit-surface claims are *advisory coordination*, not OS-enforced isolation. A malicious agent can write the file anyway (no kernel-level sandbox such as Landlock, `unveil`, or Seatbelt).
Same-user process (reads/edits the DB)	**out**	The database is owner-read/write only; the signing key is being migrated to the OS keychain. Hash-chain tamper-evidence defends against *nobody* under this exclusion (§13.4).
Different-uid process	**in (partial)**	Owner-only permissions; but the socket peer-credential check is a software handshake, not the kernel-enforced socket credential (`SO_PEERCRED`) — a real authentication caveat.
Cross-machine / cross-operator	**out (economy layer)**	Needs the complete local identity contract (PARTIAL) and cross-operator attestation (the economy paper).

13.1 Advisory claims are not sandboxing

The sharpest category error to avoid: edit-surface claims are *advisory coordination locks between cooperating agents*, not OS-enforced filesystem isolation. There is no syscall filter, no `pledge`/`unveil`, no macOS Seatbelt, no Landlock. A malicious agent ignores a claim row and writes the file. The kernel provides *advisory* coordination; real OS sandboxing of agent processes is out of scope at this layer (it belongs to the out-of-band "jail" — the separate-user kernel boundary of §13.2 — which is *proposed* and would need to be grounded in a real operating-system primitive). We say so plainly rather than let advisory claims be read as isolation.

13.2 The enforcement gate proves; it does not yet compel

The same boundary problem reappears one level up, at the kernel's *enforcement* ambition, and it is the paper's most load-bearing "*specified*, not *implemented*" caveat. The kernel ships a real cryptographic gate — a **macaroon** [25] discharge primitive (a bearer credential whose authority can only ever *narrow*, since each caveat is folded into a chained keyed hash so that tampering breaks the signature). Its reference implementation is `core/kernel/pd-anchor/src/macaroon.rs`: the root key never leaves the daemon, the third-party "rent-paid" caveat is an HMAC commitment to a discharge the daemon alone can issue, and the discharge construction is machine-checked in a protocol-verification tool (`analyses/macaroon_discharge_v1.pv` proves no push is authorized without a daemon-issued discharge bound to *that* grant). The cryptography is **implemented** and sound *as cryptography.*

> **Key idea.** A gate that is cryptographically unforgeable is not, for that reason, *compulsory.* The macaroon proves that a given push was authorized; it does not yet *stop* an unauthorized one, because the agent runs as the same user and holds its own host credentials.

What it does *not* yet do is bind that gate to the agent's actual capability to act. Today's coordination enforcement is the **Coordination Guard** (`cli/commands/guard.ts`) and a **git shim** (`cli/utils/git-shim.ts`) that intercept destructive verbs — and both run *in-band*, as the same user, through a `PATH` the agent controls. By construction that arrangement is advisory: an agent that hits the wall has at least five escapes that defeat no cryptography (a documented bypass environment variable, an absolute path to the real `git`, a `PATH` rewrite, in-process `libgit2`/editor bindings, or simply never attaching a session). The design record for this is **ADR-0053** (`docs/adr/0053-out-of-band-enforcement.md`), which states the ceiling plainly: *you cannot out-trick a same-user process from inside that same process.* The cryptographic version of the same limit is ADR-0050's concession — *a secret a process can use, it can copy* — and it applies to enforcement verbs no less than to keys: a guard a process can decline to invoke is advisory.

Closing this gap is a layered program, and only the top layer is a real wall. A server-side branch-protection-plus-push-broker layer (SPECIFIED) removes the most painful direct-to-protected-ref incidents but not same-user trickery; a credential broker that meters and injects credentials per request (*proposed*) narrows the ambient-token surface; and only a **separate-user (or separate-VM) kernel process** with forced egress (*proposed*) actually confines a malicious same-user holder, because it is the first layer that removes the capability rather than refusing the verb. That separate-user boundary is the analogue of the hardware/operating-system boundary a classical reference monitor relies on for "always invoked," and *it does not yet exist.* Until it does, the honest grade of kernel enforcement against a non-cooperating same-user agent is *not guaranteed*: the gate is a verifiable transcript and a loud tripwire, not a wall. We state this here so no reader carries the (legitimate) "mechanically verified macaroon" result forward into the (false) belief that the kernel can presently *compel* a same-user agent through it.

13.3 Partial actor-soul identity bounds every other invariant

The reference system now mints an opaque actor-soul id and lookup credential inside the daemon, and the budget guard assigns fresh actors to a shared bounded newcomer pool (I12 is PARTIAL). Legacy session paths and security-relevant writes that do not yet require that credential remain impersonable. This still bounds the security weight of I7, I8, and I9: lock-owner validity, capability escalation, note monotonicity, and owner-scoped release are only as strong as their least-protected identity path. This dependency — complete I12 enforcement gates the soundness of I7, I8, and I9 — is a hard precondition, stated here so the reader does not over-trust the enforcement organ. It is also the central reason the cross-machine layer, where identities genuinely conflict, must make identity cryptographic.

13.4 Hash-chain tamper-evidence and OS-Level Ephemeral Namespaces (OP-9)

The hash chain (I9) is built, but under the initial threat model it defended against nobody: its only adversary was someone who could edit the audit log, which was exactly the same-user process the model excluded. Tamper-evidence that nothing on the read path verifies is a data structure, not a control. We therefore re-scope I9 as **cross-machine provisioning**: it matters only against cross-machine synchronization or a tamperer who is not the same user.

To actually solve the same-machine adversary vulnerability (OP-9) locally, we introduce **OS-Level Ephemeral Namespaces**. The central `agentsd` process is isolated to run as user `0600`. When spawning agents, it executes them within `bwrap` or `unshare` sandboxes, assigning each an ephemeral UID. IPC sockets strictly enforce `SO_PEERCRED` validation, guaranteeing that the daemon can cryptographically bind incoming requests to a specific, sandboxed spawn instance, making same-user filesystem bypasses impossible.

13.5 Information Flow Control (IFC) and the Compute-to-Data Airlock

Capability scoping alone (such as granting an agent `fs:read:/confidential` and `net:egress:github.com`) is insufficient to prevent exfiltration: an agent granted both can read a confidential proprietary database and push the contents to an external server. The kernel resolves this via **Dynamic Taint Analysis** coupled with complete OS-level mediation.

The Compute-to-Data Airlock protocol. To allow Alice to license a proprietary agent stack S (compiled Wasm or opaque container) to execute over Bob's confidential data D without mutual data exfiltration:

1. **Isolated execution jail.** Bob's daemon spawns Alice's agent inside a sealed OS namespace (Linux user namespace, Landlock/cgroups, or macOS Seatbelt) with network interfaces disabled (`net:egress:none`).
2. **Dynamic taint propagation.** When the agent reads any file tagged with a confidentiality label (e.g., `fs:read:/repo/finance.csv`), the kernel hypervisor tags the agent process ID and its memory context with `[TAINT: HIGH_CONFIDENTIAL]`.
3. **Contagion & proxy DLP.** Any spawned sub-agent or shared memory write inherits the taint label. When the agent queries an LLM inference proxy, the proxy runs a local Data Loss Prevention (DLP) pattern scan over the prompt payload.
4. **Egress firewall.** If a tainted process attempts raw network egress or unauthorized file writes, the hypervisor intercepts the syscall, immediately sends `SIGKILL`, and slashes the execution bond.

This establishes an engineered DMZ — and the claim must be stated at exactly its strength. Taint propagation and the egress firewall are the *containment layer*; the enforceable security property is that **every release channel is explicit, gated by the declassification policy, and bounded by its leakage budget**. Nothing here makes exfiltration mathematically impossible (timing, ordering, and side channels remain out of model); what it makes true is that any flow of Bob's data to the outside passes a named gate whose releases are logged, budgeted, and attributable — while Alice's proprietary weights and prompts remain outside Bob's inspection surface.

Exercises.
(check) The hash chain defends against nobody in scope. Name the *out-of-scope* adversary it does defend against, and the layer (substrate or economy) where that adversary becomes real.
(trace) An agent forges its actor identity to release another agent's lock. Walk the lock-owner-validity rule and show exactly where I12's absence lets the forgery through.
(open, ★ OP-9) What is the minimal hardening that closes the same-machine adversary without a hardware trusted-platform-module dependency — an OS keychain for the signing key, sealed memory? Where does each stop?

14 Adjacency contract: what the kernel assumes and provides

The kernel's coherence with the layers around it is a contract: what it assumes from the machine below, what it provides to the protocol above, and — just as important — what it explicitly does *not* provide so higher layers do not over-assume.

14.1 What the substrate assumes from the machine

- A POSIX filesystem with working advisory locking (`flock`/`fcntl`). **This is a correctness precondition for I2**: networked filesystems break advisory locks, and a broken lock lets two writers both win — destroying mutual exclusion, not just performance.
- A local wall clock. Single-machine, so no shared-clock assumption is needed — but note that a wall clock is *not monotonic* (it steps when the clock is adjusted, e.g. by network time synchronization), an intra-node hazard for every expiry sweep that the inter-node ordering of Lamport [13] does not cover.
- Unix domain sockets with peer-credential authentication (both transports).
- An OS keychain for signing keys; file permissions honored on the database path.
- A supervisor process to keep the daemon always-on and to restart *the daemon itself* — the substrate makes other things always-on but cannot make *itself* always-on; that is delegated downward.

14.2 What the substrate provides to the coordination layer

- **Atomic, idempotent, time-to-live'd claims** (I2–I4) over ports, file-regions, symbols, and named locks — the coordination layer's typed ownership reduces to claim and release.
- **A durable, versioned, history-cursored publish/subscribe bus**, a tuple space, and decaying stigmergic markers — the coordination layer's speech act is carried *inside* the bus envelope; the anti-blackboard-rot property is *provided* by substrate-side decay.

- **The deontic split as a primitive** (Def. 8.1): prohibition → monitor (detect/regiment), obligation → commitment, permission → capability. The coordination layer must not re-implement enforcement.
- **Daemon-owned deadlines and oracle-bound closure** (I5, I6).
- **An append-only audit log** (hash-chain-provisioned) that the protocol and the operator read.
- **Self-attestation** (I10) — higher layers may *assume the kernel is honest about its own health.*
- **The cryptographic authorization chain** — the coordination layer's coordination lineage rides inside it.
- **Linearizable claim/lock semantics** (Theorem 11.1) — the property that lets the coordination layer treat "who holds this" as ground truth.

14.3 The explicit non-provisions

- The substrate does *not* provide end-to-end non-forgeable identity yet. I12 is PARTIAL: daemon-minted actor souls, lookup credentials, and a bounded newcomer pool enforced by the budget guard ship; universal write-boundary enforcement does not.
- It does *not* provide fair/queued exclusion (OP-1), cross-organ transactional atomicity by default (§12), a real execution-checkpoint (OP-4), or any cross-machine guarantee (the economy layer — different theorems, a shared-clock and Byzantine-membership problem the substrate deliberately never touches).
- It does *not* decide *what to do* (no orchestration). It enforces what *cannot* be done and records what *did* happen — a regulator, not a planner.

15 Related work

The kernel sits at the confluence of four literatures. We position it against each, and against the design space it deliberately declines.

15.1 Reference monitors and the trusted computing base

The organizing idea is the **reference monitor** of Anderson's 1972 security study [1]: a mediator that is *always invoked* (complete mediation), *tamper-resistant*, and *small enough to be verified.* Lampson's *Protection* [2] gives the access-matrix model the monitor enforces, and Saltzer and Schroeder's design principles [3] — economy of mechanism, fail-safe defaults, complete mediation — are the yardstick we hold the kernel to. Where a classical security kernel mediates memory and CPU access for an operating system, ours mediates *coordination* state for a set of cooperating processes, and it achieves complete mediation not by interposing on every system call but by the cheaper expedient of routing every mutation through a single writer. Schneider's theory of enforceable security policies via execution monitors [4] supplies the boundary that the central correction of this paper rests on (Theorem 8.1): an execution monitor can enforce only the safety properties it can prevent by truncating execution, which a post-commit observer cannot do.

15.2 Durable storage and transaction processing

The durability analysis is grounded in Gray and Reuter's canonical treatment [6], which separates atomicity and consistency from durability — the distinction the I1a/I1b split makes operational. The write-ahead-logging discipline descends from ARIES [7]; the specific crash semantics we

inherit are SQLite's WAL [8], whose own documentation states that the *normal* synchronization level may lose durability under power loss while preserving consistency. Our architectural posture — a single writer rather than a replicated state machine — follows the single-leader default that Kleppmann argues for [9]: reach for consensus only when the problem genuinely forces it. We make that argument formal via the consensus impossibility of Fischer, Lynch, and Paterson [11]: there is no agreement to reach, so the impossibility never applies. Linearizability, the external guarantee we claim for claims and locks, is Herlihy and Wing's [10]. Cross-organ atomicity, where we decline the distributed solution, is exactly the setting Garcia-Molina and Salem's Sagas [15] were designed for — and exactly the setting where, because every step is local, a single transaction dominates a saga.

15.3 Coordination, stigmergy, and deontic control

The communication organ draws on two coordination traditions. Stigmergic markers with decay descend from Grassé's stigmergy [16] and its computational realizations in ant-colony optimization [18]; the shared associative store is Gelernter's Linda tuple space [17]. The obligation arm models commitments as Cohen and Levesque's persistent goals [14] — intentions an agent may rationally drop — and the prohibition arm uses the normative-systems framing of Jones and Sergot [5] to keep prohibition, obligation, and permission distinct modalities rather than one undifferentiated "policy." The failure detector underlying heartbeat-based liveness is Chandra and Toueg's [12], which is why heartbeat freshness can never be a perfectly synchronous pre-commit check. The self-attestation organ, which denies service when its own integrity is unknown, is in the spirit of autonomic computing's self-management properties [21], and the bounded busy-wait under contention is the timeout-budget bulkhead that Nygard catalogs as a stability pattern [24].

15.4 Capability security and tamper-evident logs

The capability/permission distinction — *ability* versus *normative status* — is the object-capability discipline applied to coordination: an agent's having a handle to an operation does not make the operation permitted. The cryptographic authorization chain is a hop-bound, attenuation-monotonic delegation chain of digital signatures over an elliptic-curve signature scheme [22], of the kind whose secrecy and integrity properties are mechanically checkable in protocol-verification tools [23]; its full treatment belongs to the economy paper. The audit log's tamper-evidence is the digital-timestamping construction of Haber and Stornetta [20], built on Merkle's hash trees [19] — a structure that famously predates, and is independent of, blockchains.

> **Key idea.** The kernel is novel less in any single primitive than in their *composition under one constraint*: a single local writer turns mutual exclusion, serializability, linearizability, complete mediation, and a durable audit log into consequences of one architectural choice rather than five separately engineered subsystems. The contribution is the reduction, and the discipline of saying exactly where the reduction stops paying.

16 Open problems

These eleven open problems are the layer's research frontier; they appear as starred exercises throughout and are collected here. OP-4 is shared with the personhood paper (this paper owns the kernel mechanism; that one owns the personhood-and-reputation argument).

★ OP-1 — Fair exclusion without a scheduler.
Add bounded-wait to claims and locks while keeping the single-writer, no-background-scheduler simplicity. Is a FIFO wait-list of time-to-live'd reservations enough?

★ **OP-2 — Regimentation vs. enforcement boundary.**
Formalize which monitor rules are truly regimentable (rejected at insert) versus only enforceable (detected after). The deontic heart of the layer (Theorem 8.1).

★ **OP-3 — Cross-Runtime Soundness.**
The CI pipeline runs Differential Fuzzing: 1M concurrent operations are blasted against both test and deployment SQLite bindings, asserting strict hash equivalence of the resulting database state.

★ **OP-4 — Checkpoint with teeth.**
Realized via **Event-Sourced Neural Rehydration**. By restoring the Git SHA, truncating the JSON message array, and replaying via Prompt Prefix Caching, the daemon restores the full KV-Cache state, turning "recovery passes notes" into "recovery restores work."

★ **OP-5 — Oracle completeness.**
Solved via **Cryptographic State-Machine Assertions (CSMA)**. The oracle vocabulary now includes Delta Oracles (pre/post benchmark timing) and AST Assertions (via `tree-sitter` structural code validation), capturing real completion conditions without re-admitting free text.

★ **OP-6 — Tamper-evidence on the read path.**
Where should hash-chain verification gate? A sampling/checkpoint regime with a provable detection-probability bound (couples to §13.4).

★ **OP-7 — Schema evolution without a sequencer.**
Safe renames/backfills/down-migrations preserving "any module, any order."

★ **OP-8 — Stigmergic decay calibration.**
The right per-kind decay so coordination markers neither rot nor evaporate before they coordinate.

★ **OP-9 — Same-machine adversary.**
Solved via **OS-Level Ephemeral Namespaces**. `agentsd` runs as user `0600`; spawns execute in `bwrap`/`unshare` sandboxes with ephemeral UIDs, and IPC sockets strictly enforce `SO_PEERCRED`.

★ **OP-10 — Per-write-path durability.**
Implemented via **Selective Checkpointing**. High-stakes, money-bearing writes trigger `PRAGMA wal_checkpoint(FULL);` to achieve power-loss durability (I1b), while standard throughput operations continue without synchronous flushes.

★ **OP-11 — Cross-organ atomicity by default.**
Wrap multi-organ operations in one local transaction at the interface boundary, eliminating the undefined partial-failure semantics of §12.

17 Conclusion: solid where local, provisional where it must grow

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database, and its two jobs — durable record and mediated exclusion — are real and, mostly, proven. It earns its strongest claims (mutual exclusion, serializable/linearizable consistency, oracle-bound closure) precisely by refusing the distributed-systems problem it superficially resembles: one writer, one file, one decider, no consensus.

Where it stops, it stops formally. Durability is split: process-crash durable (I1a), *not* guaranteed against power loss (I1b). The policy monitor detects and compensates; it does not

regiment, and we credit the only genuine pre-commit regimentation to the uniqueness constraint and the boot-admission gate. Cross-organ atomicity is a buildable defect, not a frontier. Hash-chain tamper-evidence is re-scoped to cross-machine provisioning, where it actually defends someone. And the two genuinely soft spots — partial local identity enforcement (I12) and a checkpoint with teeth (OP-4) — are exactly the two things a cross-machine economy and a durable notion of agent personhood lean on hardest.

> *The kernel is solid where it is local and provisional exactly where a cross-machine economy would need it to be cryptographic and continuous. That is not a flaw to hide; it is the layer boundary, stated truthfully.*

Every "single-machine / one-writer / one-clock / self-asserted-identity" simplification that makes this layer clean is precisely the assumption a later layer must pay to relax. Cross-machine capability transfer over the cryptographic authorization chain this kernel ships is where that payment begins, and it belongs to the economy layer, not here. Build the local floor first. The cryptography earns its keep when agents must trust one another across machines they do not control.

A Implementation & status

The body of this paper argues at the level of mechanisms, so that it reads as a self-contained systems paper. This appendix records the concrete grounding: the specific artifact that realizes each mechanism in the reference implementation, and an honest accounting of how mature each is. Nothing here is needed to follow the argument; it is here so the maturity grades in the body are auditable rather than asserted.

A.1 The reference implementation

The reference implementation is a single always-on local daemon written in TypeScript, run under a compiled JavaScript runtime, persisting to one SQLite database file in WAL mode. Clients — a command-line tool, agents over a tool-protocol bridge, and a binary inter-process transport — reach it over Unix domain sockets. The daemon is kept alive by the host operating system's service supervisor. The seven organs of §3 correspond to distinct modules that each self-initialize their own tables over the shared database handle, in the factory style `createModule(db)`.

A.2 Mechanism-to-artifact map

Table 5 maps each mechanism discussed in the body to the module that realizes it. The storage configuration referenced throughout is the WAL pragma set: `journal_mode=WAL`, `synchronous=NORMAL`, `wal_autocheckpoint=200`, `busy_timeout=5000`, `foreign_keys=ON`, with a clean-shutdown `wal_checkpoint(TRUNCATE)`.

Table 5: Mechanism-to-artifact map for the reference implementation. Module names are illustrative of the organization, not a stable public interface.

Mechanism (body)	Realizing module(s)	Notes
Single write connection / pragmas	database-handle module	opens and configures SQLite
Port & lock claims	service-registry, locks, session-files modules	three claim granularities
Message bus / tuple space / markers	bus, tuples, marker modules	at-least-once delivery
Policy monitor	monitor module	post-commit log subscriber
Commitments & obligation	commitment, obligation-monitor modules	two of the hardening laws shipped
Memory & recovery	session, episodic-memory, recovery modules	recovery passes notes only
Audit log & hash chain	activity-log, hash-chain modules	verification not yet on the read path
Self-attestation	attestation, invariant-registry modules	boot gate + pre-flight + watchdog
Authorization chain	delegation-chain module	mechanically verified
Enforcement gate (macaroon discharge)	kernel macaroon module	crypto verified; in-band, not yet compulsory (§13.2)
Runtime-parity shim	runtime-shim module	normalizes binding differences
Identity (actor souls + credentials)	actor-souls, budget-guard modules	bounded gate ships; full write-boundary enforcement absent

A.3 Status, and the three load-bearing corrections

The maturity grades from Table 3 reflect the state of the reference implementation at the time of writing. Three corrections this paper makes to earlier, more optimistic internal descriptions are load-bearing and worth restating with their evidence:

1. **Durability is split by fault class** (§5). An internal code comment justifying the `synchronous=NORMAL` choice asserted that the normal level "is safe in WAL mode because WAL already guarantees crash safety." That conflates crash-consistency with durability; under power loss the last writes can roll back. The honest statement is the I1a/I1b split.

2. **The policy monitor is detect-and-compensate, not regimentation** (§8, Theorem 8.1). The monitor is implemented as a subscriber to the already-committed operation log, so it cannot prevent the bad prefix that triggers it; "physically unreachable" is reserved for the uniqueness-constraint and boot-gate states.

3. **Cross-organ atomicity is a buildable defect, not a research frontier** (§12). The database transaction primitive is already used elsewhere in the implementation; the only missing work is wrapping the multi-organ endpoints in it.

A.4 Nomenclature

For the reader cross-referencing the accompanying chapters, the body's neutral names map to the series' internal terms as follows: the *substrate / coordination / legibility / economy* layers are the series' L0–L3; the *policy monitor* is the Arbiter; *stigmergic markers* are pheromones; the *bus* is tube; the *authorization chain* and *coordination lineage* are the two objects the series

keeps distinct under the single phrase "delegation chain." The maturity grades — implemented / partial / specified / proposed — are the series' honest labels.

References

[1] James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.)

[2] Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974.

[3] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[4] Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

[5] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science*, Wiley, 1993.

[6] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. (Atomicity/consistency vs. durability — the ground for I1a/I1b.)

[7] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[8] D. Richard Hipp et al. SQLite Write-Ahead Logging. SQLite Documentation, 2010. `https://www.sqlite.org/wal.html` (`synchronous=NORMAL` "commits might lose durability following a power loss.")

[9] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. (Single-leader as the right default; consensus only when forced.)

[10] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

[11] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2):374–382, 1985.

[12] Tushar Deepak Chandra and Sam Toueg. Unreliable Failure Detectors for Reliable Distributed Systems. *Journal of the ACM*, 43(2):225–267, 1996.

[13] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978.

[14] Philip R. Cohen and Hector J. Levesque. Intention Is Choice with Commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990.

[15] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, 1987.

[16] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles…: la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959.

[17] David Gelernter. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.

[18] Marco Dorigo and Gianni Di Caro. The Ant Colony Optimization Meta-Heuristic. In *New Ideas in Optimization*, McGraw-Hill, 1999.

[19] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87*, 1987.

[20] Stuart Haber and W. Scott Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99–111, 1991.

[21] Jeffrey O. Kephart and David M. Chess. The Vision of Autonomic Computing. *IEEE Computer*, 36(1):41–50, 2003.

[22] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. (Ed25519 — the signature scheme underlying the authorization chain.)

[23] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. (The class of tool in which the authorization chain's secrecy and integrity properties are mechanically verified.)

[24] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software.* 2nd ed., Pragmatic Bookshelf, 2018. (The bounded busy-wait as a bulkhead / timeout-budget under contention.)

[25] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. (Attenuation-only bearer credentials with third-party caveats — the enforcement-gate construction.)

B Limitations & Boundaries

The formal mechanisms presented in this chapter are mathematically sound within their defined scopes, but they depend on bounding assumptions that must be explicitly acknowledged.

- **Threat Model Exclusions:** Collusion across all independent organs (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance Trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation Maturity:** While the core protocols (Anchor, SQLite single-writer) are IMPLEMENTED and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in PARTIAL or DESIGNED phases.

These constraints are not flaws but structural realities of building zero-trust coordination on top of POSIX primitives.